Production RAG Stack Forensics

A forensic study of where a production-grade retrieval pipeline breaks — and which fixes actually move the numbers. Built on LangGraph, Pinecone, Langfuse, and frontier LLM APIs over the FastAPI documentation corpus, with instrumentation exposed as a custom MCP server.

Stack: LangGraph · Pinecone · Langfuse · Claude Sonnet 4.6 / GPT-5.5 / Gemini 3.1 · FastAPI · custom MCP server | **Corpus:** FastAPI docs (584 chunks) | **Eval:** 150 questions, 5 categories, LLM-as-judge

---

What this is

This is not a tutorial and not a benchmark. It is the engineering record of building a production RAG system and measuring, honestly, where it fails. The deliverable is the autopsy: each failure mode documented with its symptom, the measurement that caught it, the mechanism behind it, the fix that was tried, and what it generalizes to. The system exists to produce those measurements.

The pipeline answers questions about the FastAPI framework from its own documentation. A query is embedded, relevant chunks are retrieved, and a language model generates an answer grounded in those chunks. Around that core sit the production tools a real team would ship on: a vector database, an orchestration framework, full tracing, an evaluation harness, and a Model Context Protocol server that exposes the project's own measurements to an AI agent.

What follows is the short version — how it was built, what broke, what I decided, and what the numbers say.

How it works

The forward pipeline has three stages, each independently measurable:

- **Retrieve.** The query is embedded (OpenAI `text-embedding-3-small`) and matched against the corpus in Pinecone. Final design uses hybrid retrieval — dense vector similarity fused with BM25 keyword matching via Reciprocal Rank Fusion.
- **Generate.** Retrieved chunks plus a grounding prompt go to the language model, which is instructed to answer only from the provided context and to say so when the context is insufficient.
- **Trace.** Every stage emits a span to Langfuse, so latency and cost are attributable per stage rather than guessed.

Evaluation runs 150 hand-curated questions across five categories — conceptual, syntactic, cross-reference, edge-case, and out-of-scope — scored by an independent LLM judge on **faithfulness** (is the answer grounded in the retrieved chunks?) and **precision@5** (how many retrieved chunks were relevant?). Questions were generated by non-Anthropic models and curated by hand, so the model being evaluated never wrote its own test.

What it solves

RAG systems pass demos and fail in production for predictable reasons: answers that look grounded but aren't, retrieval that returns the right topic but the wrong content, failures that hide behind healthy-looking averages. This project's contribution is a method for catching those before they ship — per-category measurement, an independent judge, and instrumentation that makes each pipeline stage inspectable. The four failure modes below are not hypothetical; each was found in this system and is endemic to the architecture.

The four failure modes

Faithfulness was scored 0–5. The diagnostic insight that organizes all four: **where a failure originates determines what fixes it.** Retrieval failures need retrieval fixes; generation failures need generation fixes. Applying the wrong layer's fix is wasted effort — a lesson the measurements made concrete.

Failure mode	Origin	What happens	Fix that worked
FM-1 Retrieval miss → fabrication	Retrieval	Wrong chunks returned; model fabricates a confident, grounded-looking answer	Hybrid retrieval (BM25 + dense)
FM-2 Parametric leakage	Generation	Right chunks, but model supplements with training knowledge not in the context	Few-shot grounding prompt
FM-3 Factual contradiction	Generation	Model contradicts what a retrieved chunk explicitly states	Few-shot grounding prompt
FM-4 Synthesis gap	Corpus	No single chunk contains the integration the question needs	Unfixable by retrieval (see below)

FM-4 is the one that doesn't have a clean fix, and that's the finding. When a question requires connecting two documents and the corpus simply never wrote down how they connect, no amount of retrieval tuning helps. I tested this directly: three reranking configurations — an LLM reranker, a trained cross-encoder, and a diversity-capped variant — all landed neutral-to-negative on the affected questions. Reranking can only reorder chunks that exist; it cannot manufacture missing content. The honest conclusion is that FM-4 is a corpus-structure problem masquerading as a retrieval problem, and the real solutions (cross-document chunking, knowledge graphs, agentic re-retrieval) live upstream of ranking.

The bug that nearly shipped a wrong conclusion

> **THE MOST IMPORTANT THING IN THIS PROJECT**
>
> For most of the build, every faithfulness score was computed against **empty chunk content**. The harness saved chunk filenames but stripped the chunk text. The judge was scoring whether answers *looked* grounded — rewarding citation style — not whether they actually were.

This stayed invisible for the entire project because every answer came from the same model and shared the same citation style. The judge's style-proxy correlated with real grounding only because the output distribution was uniform. The bug surfaced the moment I introduced a second model with a different output style during the cross-provider study: it scored ~2 points lower, and the implausibility of a flagship model "collapsing" on easy questions was the thread that unraveled it.

Independent confirmation came from precision@5: identical retrieved chunks were scoring differently across providers — impossible for a true retrieval metric, since retrieval is deterministic and identical regardless of which model generates the answer. That proved the judge was reading the answer to score the chunks.

I fixed the harness to preserve chunk text and re-scored. The corrected result **reversed entirely**: the model that had "collapsed" to 2.87 came back at 4.62, the highest of the three. The original number was 100% artifact. Had I trusted the healthy-looking aggregates, I would have published a confidently wrong claim about a model's quality.

The generalizable lesson: homogeneous evaluation hides judge-dependence bugs. An LLM judge scoring one model's output can pass on a proxy that only correlates with the target because the inputs are uniform. Stress-test the harness with a deliberately different model before trusting any aggregate — and verify the judge actually receives the inputs it claims to score. That is a mechanical invariant; an average cannot reveal it.

Decision tradeoffs

The interesting calls in this project were rarely "which option is best" — they were "which constraint do I accept." A few that shaped the system:

Generation-side vs. retrieval-side fixes

The few-shot grounding prompt (a generation-side change) moved faithfulness on the targeted failure records by +2 to +3 points with zero regressions. Reranking (a retrieval-side change) was neutral-to-negative. Same goal, opposite outcomes — because FM-2 and FM-3 live in generation and FM-4 lives in the corpus. Matching the fix to the failure layer was the entire difference between a clean win and wasted effort.

A negative result, kept

Reranking is an industry-default RAG improvement. On this corpus it did not earn its place, and I documented that with measurements rather than quietly dropping it. A reranker that produces confident but miscalibrated reorderings is worse than leaving deterministic dense ordering alone. Reporting the negative result honestly is a stronger signal than reporting only the wins.

Constraints I accepted, openly

- **Self-hosted Langfuse → cloud.** The local machine lacked hardware virtualization. Cloud gave the identical SDK and tracing; only the backend host changed. Documented as a deviation, not hidden.

- **Cross-provider models are not tier-matched.** Sonnet (mid), GPT-5.5 (flagship), and a Gemini flash-lite model span three tiers. The comparison is framed as "realistic per-provider production choices," never as a quality ranking — tier confounds any such ranking, and I say so.
- **precision@5 is a flawed aggregate** on a small corpus where the number of truly-relevant chunks varies per question, and is meaningless on out-of-scope questions with zero relevant chunks. I exclude out-of-scope from the aggregate and treat precision@5 as directional, not precise.

What the numbers say

Mitigations worked

Optimized stack (hybrid retrieval + grounding prompt) vs. the dense-only baseline, full 150-question set, three optimized runs to separate real movement from sample noise. Both arms were scored with the same corrected judge, so the judge is held constant across the comparison.

Metric	Baseline (dense-only)	Optimized (hybrid + grounding)	Δ
Overall faithfulness	4.57	4.82	+0.25
Confident fabrications (faith = 0)	5	0	−5
Precision@5 (answerable cats.)	0.560	0.583	+0.023

The single cleanest single-axis result is **cross-reference** — the category where FM-1 concentrated — which moved from **3.70 to 4.73 (+1.03)** going dense-only → hybrid + grounding. Confident fabrications were eliminated entirely (5 → 0).

An honest note on attribution. The optimized stack bundles two changes — hybrid retrieval and the grounding prompt — so the +0.25 aggregate gain is attributable to the stack as a whole, not cleanly split between the two. Precision@5 stayed essentially flat (+0.023, within noise) while faithfulness rose, which *suggests* the gain was generation-led (the prompt, not retrieval), since a retrieval-driven improvement would have moved precision. But that is an inference from a proxy metric — a controlled retrieval-vs-prompt ablation was not run. Reported as a suggestion, not a proven decomposition.

Cost vs. quality: the order-of-magnitude finding

Same optimized stack, generation model swapped, retrieval and judge held constant. After the judge bug was fixed:

Provider (tier)	Faithfulness	Gen cost / 150q
Anthropic Sonnet 4.6 (mid)	4.45	$2.07
OpenAI GPT-5.5 (flagship)	4.39	$5.06
Google Gemini 3.1 flash-lite (small)	4.62	$0.18

Faithfulness spans **0.23 points** — noise-level — across a **28× cost range**. The takeaway is not "Google wins" (the tiers aren't matched). It is that **generation-model capability was not the bottleneck on this corpus.** When retrieval and grounding are strong, a small, cheap model rides that quality to the same faithfulness as flagships costing an order of magnitude more. The implication for production: teams routinely over-provision generation models for RAG tasks that good retrieval has already largely solved. The boundary on the claim is honest — a harder corpus might separate the tiers; this one did not.

Latency is generation-bound

Per-stage tracing showed generation at **~91% of wall-clock** on a typical query (embed 0.5s, retrieve 0.4s, generate ~10s). This made two things concrete: the cross-provider latency story is really a generation-stage story, and the rejected LLM reranker — which added ~28 seconds of sequential API calls per query — would have quadrupled latency on an already generation-bound pipeline for no quality gain. The instrumentation turned "I think this is slower" into a number.

The MCP server: querying the system's own record

The final piece exposes the project's instrumentation as agent-queryable tools over the Model Context Protocol — six tools spanning three data sources: the engineering docs, the evaluation results, and the live Langfuse traces. Connected to a desktop AI client, it closes a recursive loop: an agent can interrogate the forensic record of the AI system through the same kind of tool interface production systems expose.

Getting it working surfaced a textbook async bug worth noting, because the debugging was the point. The Langfuse-backed tools hung indefinitely when called. The cause was not credentials or the query — isolating the call in a terminal showed it returning in 1.15s. The synchronous network call was blocking the server's async event loop, so the response never made it back over the protocol channel. Wrapping the blocking I/O to run off the event loop fixed it. Systematic isolation — terminal vs. subprocess, fast vs. infinite — pinpointed the cause before any fix was applied.

What I'd do differently from the start

The project is honest about its own seams, and the retrospective is part of the record:

- **Design the judge to read chunk content from day one.** The empty-chunk bug existed for most of the build. Verifying the judge's inputs as a mechanical invariant — not trusting its aggregates — would have caught it immediately.
- **Tier-match the cross-provider models up front,** or commit to "realistic production choices" deliberately rather than discovering the tier confound mid-study.
- **Treat evaluation infrastructure as a system to be tested,** not a measuring stick to be trusted. The most important bug in this project was in the measurement, not the thing measured.

Full five-section writeups for each failure mode, the complete engineering journal, and the locked architecture decisions are maintained in the repository's `docs/` directory. This document is the condensed record. — Cody Lee · codylee.tech